ATTENTION IS ALL YOU NEED

A complete, simple, accurate guide from absolute beginner to engineering level

FOUNDATIONS • MATHEMATICS • ARCHITECTURE • TRAINING • EXPERIMENTS • ENGINEERING

TEXT → TOKENS → EMBEDDINGS → POSITION

ATTENTION → FEED-FORWARD NETWORK → REPEATED LAYERS

ENCODER + DECODER → PROBABILITIES → GENERATED TOKENS

Original paper: Ashish Vaswani et al. | NeurIPS 2017 | arXiv:1706.03762

Teaching report edition • 12 October 2026

Executive summary

"Attention Is All You Need" introduced the Transformer, a sequence-to-sequence neural architecture that replaces recurrent and convolutional sequence-processing layers with attention-based processing. Its core operation lets each token position compare a learned query with learned keys, convert those scores into normalized weights, and use those weights to combine value vectors. Repeated attention and position-wise feed-forward layers build contextual representations.

The paper's contribution is not merely the attention equation. It is the design of a complete encoder-decoder architecture around attention: multi-head attention, causal masking in the decoder, cross-attention from decoder to encoder, sinusoidal positional encodings, residual connections, layer normalization, feed-forward networks, and a practical training recipe. On WMT 2014 machine translation, the published proceedings report 28.4 BLEU for the big model on English-to-German and 41.0 BLEU on English-to-French. The longer arXiv version includes additional parsing results and attention visualizations. [1, 2]

The central idea

A Transformer repeatedly updates token representations by letting positions retrieve and blend information from other permitted positions. This allows training computations across many positions to be performed in parallel, while autoregressive token generation still proceeds step by step.

What this report covers

- Beginner foundations: tokens, vocabularies, vectors, embeddings, neural networks, training, loss and optimization.
- Attention from first principles: queries, keys, values, dot products, scaling, softmax, weighted sums and a fully worked toy example.
- The original Transformer: multi-head attention, sinusoidal positions, encoder, causal decoder, cross-attention, residual paths, normalization and FFNs.
- Training and experiments: datasets, tokenization, hardware, optimizer, learning-rate schedule, dropout, label smoothing, BLEU, perplexity and ablations.
- Engineering: tensor shapes, masks, numerical stability, implementation pseudocode, complexity, debugging checks and common misunderstandings.
- Historical context: what BERT, GPT and T5 inherited from the Transformer and what belongs to later research.

How to use this report

A staged reading path

Stage	Read	Goal
1. Intuition	Parts I–II, especially the classroom analogy and numerical example	Explain attention without relying on jargon.
2. Architecture	Part III	Trace one source sequence through the encoder and decoder.
3. Paper fidelity	Part IV	Quote the original architecture, settings and results accurately.
4. Engineering	Part V	Reason about shapes, masks, compute, numerical stability and debugging.

Stage	Read	Goal
5. Wider context	Part VI and appendices	Connect the 2017 model to later Transformer families and test your understanding.

Evidence labels

Label	Meaning
[ORIGINAL PAPER]	A design choice, method or result reported in the 2017 NeurIPS proceedings paper.
[ARXIV VERSION]	Material available in the longer arXiv version, including appendix material and parsing results.
[BACKGROUND]	General machine-learning or NLP knowledge needed to understand the paper.
[TOY EXAMPLE]	Invented numbers used to make a mathematical operation understandable; not a published experiment.
[INTERPRETATION]	An explanatory interpretation of a mechanism, not a separate claim made by the authors.
[ENGINEERING]	Practical implementation considerations, including some conventions common in modern software.
[LATER DEVELOPMENT]	Work that followed the original paper; not a feature of the 2017 architecture.

Version policy

This report uses the NeurIPS 2017 proceedings PDF as the authority for the published paper's benchmark table. The proceedings report 41.0 BLEU for the big English-to-French model. The current arXiv v7 abstract says 41.8, while its results paragraph says 41.0; this report flags that inconsistency and does not silently merge versions. [1, 2]

Contents at a glance

- PART I — Foundations: 1. The story in one minute; 2. Text, tokens and vocabularies; 3. Vectors, matrices and embeddings; 4. Neural networks and training; 5. Sequence modeling before the Transformer.
- PART II — Attention from first principles: 6. The classroom analogy; 7. Q, K and V; 8. The scaled dot-product equation; 9. A complete numerical example; 10. Multi-head attention; 11. Position information.
- PART III — The complete architecture: 12. Encoder; 13. Decoder and causal masking; 14. Cross-attention; 15. Output probabilities; 16. Training versus generation; 17. Whole-model walkthrough.
- PART IV — Training and evidence: 18. Complexity analysis; 19. Data, hardware and optimization; 20. Translation results and decoding; 21. Ablation studies; 22. Parsing and attention visualizations; 23. What the experiments do and do not establish.
- PART V — Engineering guide: 24. Tensor shapes; 25. Masks and numerical stability; 26. Implementation skeleton; 27. Debugging checklist; 28. Limitations and design trade-offs.
- PART VI — Context and mastery: 29. BERT, GPT and T5; 30. Common misunderstandings; 31. Learning path; 32. Self-test with answer guide.
- APPENDICES — Formula sheet; historical base/big configuration; glossary; references and version notes.

PART I

Foundations: from text to learnable numerical representations

A guided progression from intuition to implementation

1. The entire story in one minute

Imagine that a computer must translate “I love cats.” A person recognizes words and grammar. A neural network instead works with numerical representations. A tokenizer first splits the text into tokens; a vocabulary maps those tokens to integer IDs; an embedding table maps each ID to a vector; and position information is added so the model can reason about order. The Transformer repeatedly mixes information between positions with attention and transforms each position with a feed-forward network. For translation, an encoder builds source representations and a decoder generates target tokens.

```
Raw text
 ↓
Tokenizer → tokens → token IDs
 ↓
Embedding vectors + position information
 ↓
Transformer layers: attention + feed-forward networks
 ↓
Output logits → softmax probabilities
 ↓
Choose / decode a token → repeat when generating
```

Remember this first

A token is a unit of text processing; an embedding is a learned vector for that token; attention determines how to combine information from other positions; the full Transformer repeats these operations in a carefully designed network.

2. Text, tokens and vocabularies

2.1 What is a token?

A token is a piece of text processed as one unit by the tokenizer. A token can be a whole word, a subword, punctuation, whitespace-associated text, a byte-level unit or a special control symbol, depending on the tokenizer. “One word equals one token” is not a general rule.

```
Teaching example (not a claim about a specific tokenizer):
Text:     I love cats.
Tokens:   [I] [love] [cats] [. ]
```

2.2 Token IDs are labels, not meanings

A tokenizer maps each token to an integer ID. The ID is an index into a vocabulary, not a numeric definition of the word. For example, a toy vocabulary might assign I → 10, love → 25, cats → 81 and punctuation → 7. The sequence becomes [10, 25, 81, 7]. The fact that “cats” has ID 81 does not mean that 81 measures how cat-like the word is.

2.3 Vocabulary and subword tokenization

A vocabulary is the set of token types available to the tokenizer. Subword methods such as Byte Pair Encoding (BPE) reduce the need to store every complete word as a separate entry. Related words can often reuse learned pieces, while rare words can be represented as a sequence of pieces. The exact split is tokenizer-specific. [1, 8]

In the original Transformer experiments, English-to-German used a shared source-target BPE vocabulary of about 37,000 tokens. English-to-French used 32,000 word pieces. These are experiment settings from 2017, not universal vocabulary sizes for Transformers. [1]

2.4 Why tokenization matters in engineering

- Token count determines sequence length, which affects memory and attention computation.
- Different tokenizers can yield different lengths for the same text, changing training throughput and inference cost.
- Special tokens may mark sequence starts, ends, padding, task boundaries or other control signals.
- Tokenization is part of the model interface: the model must be trained and used with compatible token-ID conventions.

Quick definition

Tokenization converts raw text into model-readable units. Token IDs identify those units; the embedding layer later turns their IDs into learned vectors.

3. Vectors, matrices and embeddings

3.1 Vector and dimension

A vector is an ordered list of numbers. Its dimension is the number of entries it contains. A four-dimensional toy vector is [0.2, 0.7, -0.1, 0.5]. Neural networks use vectors to represent tokens, hidden states, keys, queries, values and outputs.

3.2 Matrix

A matrix is a rectangular arrangement of numbers. If three tokens each have a four-number embedding, their representations can be stacked into a 3 × 4 matrix: three rows (tokens) by four columns (features). This notation becomes important because attention is implemented with matrix multiplication.

```
X =
[0.2  0.6  0.1  0.5]  ← token 1
[0.7  0.3  0.9  0.2]  ← token 2
[0.4  0.8  0.5  0.1]  ← token 3

Shape(X) = [sequence length, representation width] = [3, 4]
```

3.3 Embedding

An embedding is a learned vector representation associated with a token ID. For a vocabulary size V and model width d_model, the embedding table can be viewed as a matrix E with shape V × d_model. Looking up a token selects one row. For the original base Transformer, d_model = 512, so each token's embedding has 512 components. [1]

$$E \in R^{(V \times d_model)}$$

The values in an embedding are learned during training. Individual dimensions usually do not have simple human-readable meanings. It is safer to think of the vector as a useful learned representation rather than a list of manually assigned definitions.

3.4 Neural networks in one equation

A neural network is a parameterized mathematical function. A very small example is y = wx + b. The weight w and bias b are parameters; training adjusts them so the function performs better on examples. A Transformer uses the same broad idea at much larger scale, with matrix-valued weights and nonlinear transformations.

$$y = wx + b$$

If x = 3, w = 2 and b = 1, then y = 2 × 3 + 1 = 7. The example is deliberately simple: it illustrates learned parameters, not a Transformer computation.

4. Neural networks, training and optimization

4.1 Training versus inference

Training adjusts model parameters from examples. Inference uses the already-trained parameters to calculate a result for new input. During training, the model's prediction is compared with a target through a loss function; gradients are computed; an optimizer updates the parameters; and the process repeats.

```
Training loop
input + target → model prediction → loss → gradients → optimizer update → repeat

Inference
input → trained model → output
```

4.2 Loss and negative log-likelihood

For token prediction, a common loss penalizes the model when it assigns low probability to the correct token. A simplified single-target expression is L = −log(p_correct). If p_correct = 0.9, the loss is about 0.105; if p_correct = 0.1, it is about 2.303. Higher loss indicates a poorer probability assignment for that target under this measure.

$$L = -\log(p_correct)$$

A training example typically has many target positions, so the implementation aggregates token losses according to the training setup, often averaging over valid target tokens. Padding and special labels may need to be excluded from the loss.

4.3 Gradients, backpropagation and optimizers

A gradient indicates how the loss changes with a parameter. Backpropagation applies the chain rule through the computational graph to calculate gradients. An optimizer uses those gradients to update parameters. A basic gradient-descent update is:

$$w_new = w_old - \eta \cdot \partial L/\partial w$$

Here η is the learning rate. Adam, used by the original Transformer, tracks moving estimates related to gradients and squared gradients to adapt updates. The 2017 paper used β1 = 0.9, β2 = 0.98 and ε = 10^{-9}. [1]

4.4 Overfitting and regularization

Overfitting occurs when a model fits its training data too closely and generalizes less well to unseen examples. Regularization techniques are used to reduce this risk. The original Transformer used dropout and label smoothing, among other training choices. They are not attention mechanisms; they help the model train and generalize.

5. Sequence modeling before the Transformer

5.1 The encoder-decoder problem

Machine translation maps a source sequence to a target sequence. An encoder-decoder model builds representations of the source and generates a target sequence conditioned on them. This general structure predates the Transformer. The original paper's specific change was to make attention-based computation the core of the sequence-processing architecture. [1]

```
English source → Encoder → source representations → Decoder → German target
```

5.2 Recurrent neural networks: RNNs

A recurrent neural network processes positions through a chain of hidden states. A simplified recurrence is h_t = f(h_(t−1), x_t): the current state depends on the previous state and current input. This creates

sequential dependencies across positions. It can make it harder to parallelize the computations within a single training sequence.

$$h_t = f(h_{(t-1)}, x_t)$$

5.3 LSTM and GRU

LSTMs and GRUs are recurrent architectures with gates designed to control information flow through the recurrent state. They can improve learning over basic RNNs, but they still retain a recurrent step-to-step dependency.

5.4 Convolutional sequence models

Convolutional models can calculate representations for many positions in parallel. However, a local convolution kernel does not directly connect every pair of distant positions in a single layer; multiple layers may be needed to let distant positions interact. The original paper compares this path-length trade-off with self-attention. [1]

5.5 Attention before 2017

Attention was not invented from nothing by the Transformer paper. Earlier sequence-to-sequence models already used attention, commonly alongside recurrent networks. The 2017 contribution was to make attention the principal sequence-transduction mechanism while removing recurrence and convolution from the core architecture. [1, 6]

PART II

Attention from first principles: the intuition, math and worked example

A guided progression from intuition to implementation

6. The classroom analogy: what does attention mean?

Imagine a sentence as a classroom. Each word is a student. When one position is being updated, it can inspect information from other positions and decide which information is useful for its current representation. A word does not necessarily select only one other word; it can combine information from several positions, assigning different weights to different sources.

For instance, in the sentence “The dog chased the cat because it was frightened,” a model may learn that information about “dog,” “cat,” “frightened” and the sentence structure is relevant when processing “it.” The answer is not guaranteed by the attention formula alone: the behavior depends on learned parameters, training data, context and the surrounding network.

Classroom analogy	Transformer quantity	Careful interpretation
What the current student is looking for	Query (Q)	A learned query vector used to compare against keys.
Matching information carried by each student	Key (K)	A learned key vector used in compatibility scoring.
Information each student can contribute	Value (V)	A learned value vector that will be mixed into the output.
How well the query matches each key	Dot-product score	A learned compatibility score, not a literal human judgment.
Relative weight assigned to each source	Softmax attention weight	A normalized coefficient used in the weighted sum.
Updated understanding of the current student	Attention output	A new vector; not necessarily a human-readable explanation.

Important limit of the analogy

Attention weights are computational coefficients. They are not automatically faithful explanations of all the reasoning a network performed, and a large weight should not be read as a complete causal explanation of a prediction.

7. Query, Key and Value (Q, K, V)

7.1 Why three representations?

The same input representation is projected into three different learned spaces because scoring relevance and carrying content are different jobs. Query and key vectors determine compatibility; value vectors carry the information that is blended into the output. “Q asks, K matches, V supplies” is a useful mnemonic, but not a literal description of conscious questions or labels.

Let X be a matrix containing token representations. The model applies learned linear projections:

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

W_Q, W_K and W_V are trainable parameter matrices. In self-attention, the Q, K and V projections are all derived from the same input sequence representation X. In cross-attention, the queries come from the decoder, while the keys and values come from encoder output.

7.2 Shape intuition

For a toy sequence with three positions and a model width of four, X has shape 3 × 4. If a single attention head uses a key/value width of two, the projection matrices can map X to Q, K and V matrices with shape 3 × 2. These dimensions are deliberately small to make the arithmetic visible.

Symbol	Role	Toy shape	Original base per head (one sequence of length n)
X	Input representations	3 × 4	n × 512
W_Q	Query projection	4 × 2	512 × 64
W_K	Key projection	4 × 2	512 × 64
W_V	Value projection	4 × 2	512 × 64
Q, K, V	Projected representations	3 × 2 each	n × 64 each

8. Scaled dot-product attention, derived step by step

8.1 The main equation

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

This is Equation (1) in the original paper. [1] Read it as a sequence of operations rather than as one mysterious formula.

1. Compute query-key compatibility scores with QK^T.
2. Divide each score by $\sqrt{d_k}$ to scale its magnitude.
3. Apply softmax across key positions to obtain weights that sum to 1 for each query.
4. Use those weights to calculate a weighted sum of the value vectors.

8.2 Dot product

The dot product of two same-length vectors is the sum of products of corresponding components. For q = [1, 2] and k = [3, 4], q · k = 1×3 + 2×4 = 11. It gives a scalar compatibility score. In a learned network, a dot product is not inherently a probability or a semantic truth score; it is a value that feeds the next steps.

$$[1, 2] \cdot [3, 4] = 11$$

8.3 Why transpose K?

Suppose Q has shape n × d_k and K has shape n × d_k. The transpose K^T has shape d_k × n. Therefore, QK^T has shape n × n: each row corresponds to one query position and each column to one key position. Each score matrix entry compares one query with one key.

$$(n \times d_k)\,(d_k \times n) = n \times n$$

8.4 Why divide by the square root of the key dimension?

The paper explains that if the components of query and key vectors are independent, have mean zero and variance one, the dot product's variance grows with d_k. Large-magnitude logits can push softmax into regions where gradients are very small. Dividing by $\sqrt{d_k}$ moderates the score scale. This is a stabilization choice justified by the paper's analysis, not a guarantee that every row will have a particular shape. [1]

If $d_k = 64$, $\sqrt{d_k} = 8$. A raw score of 16 is scaled to 2. This demonstrates the arithmetic; it is not a full probabilistic claim.

$$\text{scaled_score} = \text{raw_score} / \sqrt{d_k} \qquad \text{example: } 16 / \sqrt{64} = 16 / 8 = 2$$

8.5 Softmax

Softmax converts real-valued scores into positive values that sum to one along the normalized dimension. For scores z_i, $\text{softmax}(z_i) = \exp(z_i) / \Sigma_j \exp(z_j)$. In attention, normalization is performed across candidate key positions for each query.

$$\text{softmax}(z_i) = \exp(z_i) / \Sigma_j \exp(z_j)$$

Example: for [2, 1, 0], exp scores are approximately [7.389, 2.718, 1.000], with total 11.107. The softmax output is approximately [0.665, 0.245, 0.090]. The largest score receives the largest weight, but smaller scores still receive some weight.

8.6 Weighted sum of values

Let the attention weights be [0.4, 0.2, 0.4], with value vectors V_1 = [1, 0], V_2 = [0, 2] and V_3 = [3, 1]. The output is $0.4V_1 + 0.2V_2 + 0.4V_3$ = [1.6, 0.8]. Every value contributes in proportion to its weight. The output is a vector, not a chosen word.

$$o = \Sigma_j \, a_j \, V_j = 0.4[1,0] + 0.2[0,2] + 0.4[3,1] = [1.6, 0.8]$$

9. Complete toy numerical example: three tokens, one head

Scope and assumptions

This is a hand-calculated toy example chosen for clarity. It is not the exact 2017 model, does not use trained parameters and is not a reported experiment.

9.1 Define the input matrix and projections

Use three token positions, model width four and one head with head width two. Let:

$$X = [[1,0,1,0], [0,1,0,1], [1,1,0,0]]$$

Shape(X) = 3 × 4. Choose toy projection matrices:

```
W_Q = W_K = [[1,0], [0,1], [1,0], [0,1]]
W_V       = [[1, 0], [0,1], [1,1], [1,-1]]
```

9.2 Calculate Q, K and V

Matrix multiplication gives the following projected representations:

```
Q = X W_Q = [[2,0], [0,2], [1,1]]
K = X W_K = [[2,0], [0,2], [1,1]]
V = X W_V = [[2,1], [1,0], [1,1]]
```

All three matrices have shape 3 × 2. In this toy setup Q happens to equal K because the chosen matrices W_Q and W_K are identical; this is a simplifying choice, not a property required by Transformers.

9.3 Calculate the score matrix

First compute QK^T. Each row is one query and each column one key:

```
Q K^T = [[4,0,2],
         [0,4,2],
         [2,2,2]]
```

Since d_k = 2, divide each score by $\sqrt{2} \approx 1.414$:

```
S = Q K^T / √2 ≈ [[2.828, 0.000, 1.414],
                  [0.000, 2.828, 1.414],
                  [1.414, 1.414, 1.414]]
```

9.4 Apply row-wise softmax

Applying softmax independently to each row gives approximate attention weights:

```
A ≈ [[0.768, 0.045, 0.187],
     [0.045, 0.768, 0.187],
     [0.333, 0.333, 0.333]]
```

Each row sums to approximately 1. The first query puts most of its weight on the first key, the second query on the second key, and the third query distributes its weight uniformly because its three scaled scores are equal.

9.5 Multiply attention weights by V

The attention output is O = AV. For the first row, for example:

$$o_1 = 0.768[2,1] + 0.045[1,0] + 0.187[1,1] \approx [1.768, 0.955]$$

The complete output is approximately:

```
O ≈ [[1.768, 0.955],
     [1.045, 0.232],
     [1.333, 0.667]]
```

9.6 What the example shows

- Attention scores are computed for every query-key pair.
- Scaling occurs before softmax.
- Softmax normalizes a query's weights across key positions.
- The output is a weighted sum of the value vectors.
- One head with width two returns three two-dimensional vectors; the original model concatenates several heads and projects them to model width.

10. Multi-head attention

10.1 Why use more than one head?

A single attention operation produces one way of comparing queries with keys and mixing values. Multi-head attention gives the model multiple learned projection spaces. Each head can compute its own attention pattern; their outputs are concatenated and projected back to the model width. The original authors argued that this lets the model jointly attend to information from different representation subspaces and positions. [1]

$$\text{head_i} = \text{Attention}(QW_Q\text{^}i, KW_K\text{^}i, VW_V\text{^}i)$$

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head_1}, \ldots, \text{head_h})\ W_O$$

10.2 Base-model dimensions

Quantity	Base Transformer setting	Explanation
d_model	512	Width of token representations in the encoder/decoder stack.
h	8	Number of attention heads.
d_k	64	Key/query width per head.
d_v	64	Value width per head.
Concatenated head width	8 × 64 = 512	Returns to the model width before the output projection.
d_ff	2048	Inner width of the position-wise FFN.

Because d_model = 512 and there are eight heads, each head uses d_k = d_v = 512 / 8 = 64. Per head, Q, K and V have shape n × 64; the score matrix has shape n × n; the weighted values have shape n × 64. Concatenating eight head outputs creates n × 512, and W_O maps the result to model width. [1]

10.3 What does an individual head do?

A head is a learned attention calculation, not a manually assigned linguistic module. Some heads may show patterns correlated with syntax, reference resolution or long-distance dependencies, but there is no guarantee that one head corresponds exactly to one grammar rule. The longer arXiv version illustrates several such patterns and describes them cautiously. [2]

11. Position information: why and how

11.1 Why attention needs a position signal

Self-attention compares representations at positions, but the operation itself does not automatically encode the original left-to-right order. If the model had only content vectors and no positional information, reordering a sequence would not be represented through an explicit position signal. The original Transformer adds positional encodings to the token embeddings at the bottoms of both encoder and decoder stacks. [1]

11.2 Original sinusoidal formulas

$$PE(pos, 2i) = \sin(pos / 10000^{(2i / d_model)})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i / d_model)})$$

Here, pos is a token position, i indexes a pair of dimensions, and d_model is the representation width. Even dimensions use sine and odd dimensions use cosine. Different dimensions use different wavelengths, ranging geometrically from 2π to $10000 \times 2\pi$ in the paper's description. [1]

11.3 Tiny numerical example

For an invented example with d_model = 4 and pos = 1, the first pair is sin(1) ≈ 0.8415 and cos(1) ≈ 0.5403. For i = 1, the denominator is 10000^(2/4) = 100, so sin(0.01) ≈ 0.0100 and cos(0.01) ≈ 0.99995. The four-dimensional positional vector is approximately [0.8415, 0.5403, 0.0100, 0.99995].

11.4 Add position to the token embedding

The positional encoding has the same width as the embedding, so it can be added component by component. For example, [0.2, 0.5, 0.3, 0.7] + [0.8, 0.5, 0.01, 1.0] = [1.0, 1.0, 0.31, 1.7]. The real model uses learned embeddings and calculated sinusoidal position values, not these toy numbers.

11.5 Why sinusoidal?

The authors hypothesized that the fixed sine/cosine construction could help the model learn relationships at relative offsets: for a fixed offset, the encoding at position pos + k can be represented as a linear function of the encoding at pos. They also tested learned positional embeddings and found nearly identical results in the reported ablation. They chose sinusoids partly because fixed encodings might extrapolate to longer sequences than those seen during training; the paper does not prove that sinusoidal positions are universally superior. [1]

PART III

The complete original Transformer: encoder, decoder and token generation

A guided progression from intuition to implementation

12. The encoder stack

12.1 Purpose

The encoder converts a source sequence into a sequence of contextual continuous representations. If the input contains n tokens, the encoder normally produces n representations. In the original base model, the encoder contains six identical layers stacked sequentially. “Identical” means the layers share a design; their parameters are not all tied to be the same. [1]

12.2 One encoder layer

```
Input X [n, 512]
  ↓
Multi-head self-attention
  ↓
Dropout → residual add → LayerNorm
  ↓
Position-wise feed-forward network (512 → 2048 → 512)
  ↓
Dropout → residual add → LayerNorm
  ↓
Output [n, 512]
```

The model width remains d_model = 512 through the sublayers so residual additions are shape-compatible. Attention exchanges information between positions. The FFN then transforms each position independently using the same two-layer network across positions within that layer. [1]

12.3 Residual connection

A residual connection adds the input of a sublayer to that sublayer’s output. The original Transformer uses LayerNorm(x + Sublayer(x)). This preserves a direct information path across the sublayer and supports optimization of a deep stack. The precise original arrangement is post-norm: normalization occurs after the addition. Modern architectures often use other arrangements, but they should not be mistaken for the 2017 design.

$$y = \mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$$

12.4 Layer normalization

Layer normalization standardizes a vector across its normalized feature dimensions and then applies learned scale and shift parameters. A simplified formula is:

$$\mathrm{LayerNorm}(x) = \gamma \cdot (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)} + \beta$$

Here μ and σ^2 are the mean and variance over the normalized features, γ and β are learned parameters, and ε is a small numerical constant. This formula provides background intuition; implementation details such as the exact variance convention and tensor axes depend on the framework.

12.5 Position-wise feed-forward network (FFN)

The FFN uses two linear transformations with ReLU between them:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)\, W_2 + b_2$$

In the base model, it expands 512 features to 2048 and projects back from 2048 to 512. The parameters are shared across sequence positions within the same layer, but different layers have different FFN parameters. The phrase “position-wise” matters: the FFN does not itself mix information between token positions. [1]

12.6 ReLU

$$\mathrm{ReLU}(x) = \max(0, x)$$

For example, ReLU([-2, 3, -1, 5]) = [0, 3, 0, 5]. ReLU is applied element-wise between the two linear transformations in the original FFN.

13. The decoder stack and causal masking

13.1 Decoder purpose

The decoder generates the target sequence one output token at a time during inference. The original decoder also has six layers. Each layer contains masked multi-head self-attention, encoder-decoder attention, and a position-wise FFN, with a residual connection and layer normalization around each sublayer. [1]

13.2 Why masked self-attention?

While predicting a target token, the model must not use later target tokens that would be unknown at generation time. A causal mask blocks attention from a position to future positions. The paper masks illegal score entries with negative infinity before softmax; because $\exp(-\infty) = 0$, the corresponding attention weights become zero. [1]

```
Example causal mask (✓ = visible; × = blocked):
Query position      Key 1   Key 2   Key 3
1                   ✓       ×       ×
2                   ✓       ✓       ×
3                   ✓       ✓       ✓
```

The exact convention for whether position i sees the current position depends on how inputs and prediction targets are shifted. With the paper's shifted decoder input, the representation at a position can predict the next target while using only the known earlier target prefix.

13.3 Shifted target sequence / teacher forcing

During training, the correct target sequence is available. The decoder receives a one-position-shifted version and learns to predict the next token at each position. For example:

```
Decoder input:  <START> I love
Targets:        I       love cats
```

The causal mask makes it possible to calculate the target positions together in a parallel matrix operation without leaking future target tokens. This is a key difference between parallelizable training and sequential autoregressive generation.

14. Encoder-decoder attention (cross-attention)

In the decoder's second attention sublayer, queries come from the decoder representation while keys and values come from the encoder output. This allows every decoder position to retrieve information from the source sequence. The mechanism is called "encoder-decoder attention" in the original paper and is commonly called cross-attention in later literature. [1]

```
Q ← decoder hidden states
K ← encoder output
V ← encoder output
```

Intuition: the decoder asks which source information is useful for the target token currently being generated. Technically, it performs query-key compatibility scoring over the encoder positions and forms a weighted mixture of encoder value vectors. It does not need a human-provided alignment between every source and target word.

Attention type	Q comes from	K and V come from	Allowed context
Encoder self-attention	Encoder layer input	Encoder layer input	All source positions, subject to padding masks.

Attention type	Q comes from	K and V come from	Allowed context
Decoder masked self-attention	Decoder layer input	Decoder layer input	Current and earlier target positions, never future target positions.
Encoder-decoder / cross-attention	Decoder layer input	Encoder output	All permitted source positions.

15. From decoder states to output probabilities

15.1 Linear projection and logits

After the decoder stack, each target position has a model-width vector. A learned linear transformation maps it to a score for each vocabulary item. Those raw scores are logits; they need not sum to one and can be positive or negative.

```
decoder representation → linear projection → vocabulary logits → softmax → probabilities
```

15.2 Softmax output

Softmax converts logits into a probability distribution over the vocabulary. If logits for three candidate tokens were [5.2, 3.1, 1.4], their probabilities would be positive and sum to one after softmax. The example values are invented. In real translation, the vocabulary has many more than three entries.

15.3 Weight sharing

The original Transformer shares a weight matrix between the two embedding layers and the pre-softmax linear transformation. It also scales embedding weights by √d_model. These are specific implementation choices reported in the paper and are often omitted in simplified diagrams. [1]

16. Training versus autoregressive generation

16.1 Why training is parallelizable

During training, the target sequence is known. With a shifted decoder input and a causal mask, all target-position losses can be computed in parallel while each position is prevented from accessing future target tokens. The encoder also processes all source positions in parallel in the attention layers.

16.2 Why generation is still sequential

During autoregressive generation, the future target sequence is not known. The system predicts a token, appends it to the prefix, then predicts the next token. A decoder cannot produce token 5 conditioned on the actual generated token 4 before token 4 exists. Inference optimizations such as caching can reduce repeated work, but they do not make standard left-to-right generation logically independent across positions.

```
Training (parallel positions with a causal mask):
<START> I love  → predict I, love, cats at target positions together

Generation (autoregressive):
<START> → I → I love → I love cats → stop
```

Interview-ready distinction

Transformer training is highly parallelizable across sequence positions. Standard autoregressive generation remains sequential across generated tokens, even though operations within each new decoding step can be parallelized across heads and other tensor dimensions.

17. Whole-model walkthrough

Use “I love cats.” as a source example. The toy tokenization below is only illustrative; a real tokenizer can split the text differently.

1. Raw text is passed through a tokenizer.

2. The tokenizer returns a sequence of token IDs.

3. Each ID selects a learned embedding vector.

4. A positional encoding is added to each embedding.

5. The six encoder layers repeatedly apply multi-head self-attention, residual + LayerNorm, an FFN and residual + LayerNorm.

6. The encoder returns source representations for every input position.

7. The decoder starts from a shifted target prefix and applies masked self-attention.

8. The decoder uses cross-attention to access encoder output.

9. The decoder FFN transforms each position; this stack is repeated six times in the base model.

10. A linear output projection creates vocabulary logits.

11. Softmax maps logits to a probability distribution for the next token.

12. The decoding algorithm chooses or searches candidate continuations and continues until an end condition is met.

```
Source text → tokenizer → IDs → embeddings + positions → encoder
                                                  ↓
Target prefix → embeddings + positions → masked decoder self-attention
                                                  ↓
                                decoder cross-attention ← encoder output
                                                  ↓
                                FFN / residual / normalization, repeated layers
                                                  ↓
                                linear output → logits → softmax → next token
```

PART IV

Training recipe, benchmark evidence and experimental findings

A guided progression from intuition to implementation

18. Complexity: why self-attention was attractive

18.1 The paper's comparison

The paper compares layer types using three criteria: computation per layer, minimum number of sequential operations and maximum path length between distant positions. Its table is a theoretical comparison under the paper's definitions, not a guarantee of wall-clock speed for every hardware and sequence length. [1]

Layer type	Complexity per layer	Sequential operations	Max. path length
Self-attention	$O(n^2 d)$	O(1)	O(1)
Recurrent	$O(n d^2)$	O(n)	O(n)
Convolutional	$O(k n d^2)$	O(1)	O(log_k n)
Restricted self-attention	O(r n d)	O(1)	O(n/r)

In the table, n is sequence length, d is representation width, k is convolution kernel width and r is the size of a local attention neighborhood. Actual implementations also pay for Q/K/V and output projections, FFNs, data movement and framework overhead; the table focuses on the core comparison. The original authors noted that self-attention is faster than recurrence in terms of complexity when n < d, which is often true for translation sentences, but not automatically true for very long sequences. [1]

18.2 Quadratic score-matrix growth

Full self-attention constructs an n × n score matrix per head and example. Therefore the score-count grows quadratically with sequence length: n = 10 gives 100 positions; n = 1,000 gives one million; n = 10,000 gives one hundred million. With a batch and multiple heads, the score tensor shape is B × H × N × N, which can dominate memory use.

```
10 tokens      → 10² = 100 score entries per head
1,000 tokens   → 1,000² = 1,000,000 score entries per head
10,000 tokens  → 10,000² = 100,000,000 score entries per head
```

18.3 Restricted attention and the long-context issue

The paper already recognized the difficulty of very long sequences and proposed restricting attention to a neighborhood of size r as a possible direction. This reduces core attention work from quadratic n^2 to approximately rn, at the cost of a longer path to connect distant positions. Modern sparse, local, block, linearized and memory-efficient approaches extend this broad line of investigation, but those later methods should not be attributed to the original model.

19. Data, hardware and training configuration

19.1 Training datasets and tokenization

Task	Data volume	Tokenization / vocabulary
WMT 2014 English → German	About 4.5 million sentence pairs	Shared source-target BPE vocabulary of about 37,000 tokens.
WMT 2014 English → French	36 million sentences	32,000 word-piece vocabulary.

Training pairs were grouped by approximate sequence length. Each batch contained about 25,000 source tokens and 25,000 target tokens. Token-based batching helps manage the amount of computation more consistently than batching a fixed number of sentence pairs, because sentence lengths vary. [1]

19.2 Hardware and runtime

Model setting	Reported step time	Steps	Reported training time	Hardware
Base	About 0.4 seconds	100,000	About 12 hours	One machine with 8 NVIDIA P100 GPUs
Big	About 1.0 second	300,000	About 3.5 days	One machine with 8 NVIDIA P100 GPUs

These are historical measurements from the paper, not estimates for current GPUs or modern framework implementations. End-to-end training cost depends on hardware, precision, batching, software, sequence lengths and parallelism. [1]

19.3 Adam and the learning-rate schedule

The paper uses Adam with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\varepsilon = 10^{-9}$. Its learning rate follows:

$$\text{lrate} = \text{d_model}^{(-0.5)} \times \min(\text{step}^{(-0.5)}, \text{step} \times \text{warmup}^{(-1.5)})$$

The warmup period is 4,000 steps. The learning rate increases linearly during warmup and then decreases proportionally to the inverse square root of the step number. The model-width factor $\text{d_model}^{(-0.5)}$ also scales the schedule. The constants and formula are from the original paper. [1]

Conceptually, warmup makes early updates more cautious while the network parameters are still changing rapidly; the subsequent decay reduces the learning rate over training. This intuition is useful, but the schedule's reported success is empirical, not a mathematical guarantee of convergence for arbitrary models.

19.4 Dropout

The base model uses dropout probability 0.1. The paper applies dropout to sublayer outputs before residual addition and normalization, and to sums of token embeddings and positional encodings in the encoder and decoder. The English-to-French big model uses dropout 0.1 rather than 0.3, while the big setting in the model-variation table is listed with 0.3. [1]

19.5 Label smoothing

The paper uses label-smoothing value $\varepsilon_{ls} = 0.1$. Instead of assigning all target mass to a single one-hot class, label smoothing distributes a small amount of probability mass across alternatives. This regularizes the classification target. The authors report that it worsened perplexity but improved accuracy and BLEU in their experiments. [1]

20. Translation results, BLEU, perplexity and decoding

20.1 Published test results

Model	WMT14 EN→DE BLEU	WMT14 EN→FR BLEU	Reported training-cost figure
Transformer base	27.3	38.1	3.3×10^{18} FLOPs for EN→DE; EN→FR not listed in this row
Transformer big	28.4	41.0	2.3×10^{19} FLOPs for EN→DE; published table displays EN→FR training cost in the paper

The published proceedings Table 2 reports 27.3 / 38.1 BLEU for the base model and 28.4 / 41.0 for the big model on the WMT14 English-German and English-French test sets. The big English-German model outperformed earlier reported results, including ensembles, by more than two BLEU in the authors' comparison. The authors also reported that the Transformer could be trained at substantially lower computational cost than competitive systems in that comparison. [1]

BLEU version caution

The arXiv v7 abstract reports 41.8 for English-to-French, but the arXiv results paragraph and the NeurIPS proceedings abstract/table report 41.0. Use a named paper version when quoting this figure. This report uses 41.0 for the published 2017 result. [1, 2]

20.2 What BLEU measures

BLEU is a machine-translation metric based on modified n-gram precision and a brevity penalty relative to reference translations. It measures aspects of overlap with reference text. It is useful for reproducible comparisons within an evaluation setup but is not a direct measure of understanding, usefulness, factuality or human preference. A high BLEU score should be interpreted in the context of the dataset, references and evaluation protocol.

20.3 Perplexity

Perplexity summarizes how well a model assigns probability to a sequence under a language-modeling or token-prediction objective. Lower perplexity commonly corresponds to assigning higher probability to the observed targets. The original Table 3 states that its perplexities are per wordpiece under the authors' BPE setup and should not be compared directly with per-word perplexities. [1]

20.4 Beam search and length penalty

The paper uses beam search with beam size 4 and length-penalty $\alpha = 0.6$ for machine-translation decoding. Instead of keeping only the most likely next token at each step, beam search keeps several promising partial sequences and expands them. The paper averaged the last five checkpoints for base models and the last twenty for big models. It limited the output length to input length + 50 while allowing early termination. These are decoding choices reported in the paper, not mandatory settings for every Transformer. [1]

21. Ablation studies: what design choices mattered?

An ablation changes one part of a model or training recipe while keeping the other reported settings fixed, then measures the effect. The paper's Table 3 reports English-to-German development-set results on newstest2013. The baseline is the base configuration with 4.92 wordpiece perplexity, 25.8 BLEU and about 65 million parameters. Unless otherwise stated, table entries are the reported table values. [1]

21.1 Number of heads

Heads h	Per-head d_k = d_v	Dev PPL	Dev BLEU	Interpretation
1	512	5.29	24.9	Single-head attention performs worse than the base setting.
4	128	5.00	25.5	Closer to the base result.
8 (base)	64	4.92	25.8	Strong baseline.
16	32	4.91	25.8	Comparable score in this setup.

Heads h	Per-head d_k = d_v	Dev PPL	Dev BLEU	Interpretation
32	16	5.01	25.4	Quality decreases with very narrow heads here.

Takeaway: multi-head attention is useful in the tested setup, but adding more heads indefinitely does not guarantee gains. The experiment changes head count and head width together while keeping the amount of computation approximately constant. [1]

21.2 Key/value dimension

d_k = d_v	Dev PPL	Dev BLEU	Parameters	Lesson
16	5.16	25.1	58M	The smaller projection dimension hurts this setup.
32	5.01	25.4	60M	Better than width 16, but below baseline.
64 (base)	4.92	25.8	65M	Best among these listed settings.

21.3 Depth, model width and FFN width

Changed setting	Value	Dev PPL	Dev BLEU	Parameters
Number of layers N	2	6.11	23.7	36M
Number of layers N	4	5.19	25.3	50M
Number of layers N	8	4.88	25.5	80M
d_model	256	5.75	24.5	28M
d_model	1024	4.66	26.0	168M
d_ff	1024	5.12	25.4	53M
d_ff	4096	4.75	26.2	90M

Within these tested settings, more layers, wider model representations and wider FFNs generally improved the reported development performance, at increased parameter counts. The table should not be over-read as a universal scaling law: it reflects specific architecture configurations, a particular dataset and a specific training budget. [1]

21.4 Dropout, label smoothing and positional embeddings

Variation	PPL	BLEU	Paper-supported conclusion
Base: dropout 0.1, label smoothing 0.1	4.92	25.8	Reference configuration.
Dropout 0.0 (row D settings)	5.77 or 4.67	24.6 or 25.3	Removing dropout hurts in the tested comparisons.
Dropout 0.2 (row D settings)	4.95 or 5.47	25.5 or 25.7	The effect depends on which regularization setting is changed.
Learned positional embeddings instead of sinusoids	4.92	25.7	Nearly identical to the base result.

The printed Table 3 presents multiple row-D combinations for dropout and label smoothing; consult the original table when making a precise claim about any individual row. The safe high-level conclusion is that regularization mattered, larger tested models tended to do better, and learned positional embeddings performed nearly the same as sinusoidal encodings in this experiment. [1]

22. Parsing results and attention visualizations in arXiv

22.1 English constituency parsing

The longer arXiv version includes an experiment on English constituency parsing. Parsing builds a hierarchical grammatical structure for a sentence, such as noun phrases and verb phrases. This task differs from translation and stresses structured output and output sequences that can be longer than the input. [2]

The reported four-layer Transformer used d_model = 1024. In the WSJ-only setup it trained on about 40,000 Penn Treebank sentences with a vocabulary of 16,000 tokens and achieved 91.3 F1 on WSJ Section 23. In a semi-supervised setup using much larger corpora (approximately 17 million sentences), it used a 32,000-token vocabulary and achieved 92.7 F1. These values belong to the arXiv version's additional table, not the core NeurIPS proceedings version being used for the main benchmark discussion. [2]

22.2 F1 in simple terms

Precision asks what fraction of predicted structure units are correct; recall asks what fraction of reference structure units the system found. F1 is their harmonic mean:

$$F1 = 2 \times (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$$

22.3 Attention visualizations

The arXiv appendix contains visualizations of attention heads. Examples include heads attending across a long-distance dependency associated with "making … more difficult," and heads whose patterns appear related to anaphora or pronoun/reference resolution, including the token "its." The authors also note that different heads seem to perform different tasks and that some patterns correlate with sentence structure. [2]

How to describe these plots accurately

Say that some attention heads exhibit patterns associated with particular linguistic relationships in the displayed examples. Do not claim that every head maps cleanly to a grammar rule or that an attention map fully explains the model's internal reasoning.

23. What the experiments establish—and what they do not

23.1 What the paper demonstrates

- The attention-based encoder-decoder architecture can achieve strong machine-translation quality on the paper's WMT14 benchmarks.
- The Transformer enables more parallelizable training than sequence-processing architectures built around recurrent dependencies.
- The reported ablations show that head count, key/value dimensions, model size, regularization and positional-encoding choice can affect quality.
- The longer arXiv version provides evidence of transfer to constituency parsing and includes illustrative attention patterns.

23.2 What it does not prove

- It does not prove that attention solves every sequence task or that full attention is efficient for arbitrarily long sequences.
- It does not make autoregressive generation fully parallel across future tokens.
- It does not prove sinusoidal positional encodings are universally better than learned or later positional methods.
- It does not show that attention weights are always faithful explanations of model reasoning.
- It does not describe the complete training, alignment, safety or inference recipe of modern large language models.

PART V

Engineering guide: tensor shapes, masks, numerical stability and debugging

A guided progression from intuition to implementation

24. Tensor shapes in implementation

24.1 Name your dimensions

A common implementation uses X with shape [B, N, D], where B is batch size, N is sequence length and D is model width. Let H be the number of heads and D_h the width per head. In the original base model, D = 512, H = 8 and D_h = 64, so D = H × D_h.

Symbol	Meaning	Example base setting
B	Batch size	Chosen by training/inference configuration
N	Sequence length	Varies by input
D or d_model	Model width	512
H or h	Number of heads	8
D_h or d_k = d_v	Per-head width	64
d_ff	FFN inner width	2048

24.2 Q/K/V and score shapes

```
Input: X [B, N, D]
Project / split heads: Q, K, V [B, H, N, D_h]
Transpose K for scoring: K^T [B, H, D_h, N]
Scores = Q @ K^T: [B, H, N, N]
Weights = softmax(scores / sqrt(D_h), dim=-1): [B, H, N, N]
Head output = weights @ V: [B, H, N, D_h]
Transpose + concatenate heads: [B, N, H * D_h] = [B, N, D]
Output projection: [B, N, D]
```

Softmax must normalize over the key-position axis for each query. In the standard shape shown above, that is the final dimension of the [B, H, N, N] score matrix. Padding masks must be shaped and broadcast according to the framework's conventions, and the causal mask must block future target positions.

24.3 Computation and memory

The [B, H, N, N] score/weight tensor is a common memory hotspot. Increasing sequence length from N to 2N multiplies the entries in this tensor by four. Head count and batch size also multiply its size. An implementation may fuse operations, recompute selected intermediates or avoid materializing full attention weights to reduce memory traffic, but such optimizations are engineering developments beyond the original paper's mathematical definition.

25. Masks, numerical stability and practical correctness

25.1 Causal and padding masks are different

A causal mask prevents a target position from attending to later target positions. A padding mask prevents the model from assigning attention to padding positions that do not represent real input tokens. Some tasks need both kinds of masks, while encoder self-attention typically needs padding handling but not a causal restriction. Cross-attention usually needs a source-padding mask.

25.2 Masking before softmax

The original paper describes setting disallowed score entries to negative infinity before softmax. In actual software, use the framework's documented attention-mask interface or a numerically safe sentinel

appropriate to the data type. Check that forbidden positions receive effectively zero weight and that valid rows do not end up with all positions masked.

25.3 Stable softmax

A numerically stable softmax typically subtracts the row maximum before exponentiation. Mathematically, softmax(z) equals softmax(z − c) for any scalar c applied to every component in the row, so subtracting the maximum does not change the distribution but helps avoid overflow. Many library implementations do this internally.

25.4 Shape and semantic invariants

- d_model must be divisible by the number of heads when the implementation uses equal-width heads.
- Q and K must have matching per-head feature widths; the sequence dimensions must align for QK^T.
- The attention output's sequence axis must align with the residual branch before addition.
- Softmax weights over valid key positions should sum to approximately one for each query, apart from all-masked edge cases.
- The FFN operates independently at each position and returns to d_model width.
- The decoder's training targets must be shifted consistently with the causal mask.
- The tokenizer, vocabulary, embedding matrix and output vocabulary mapping must agree.

26. Minimal implementation skeleton

The following PyTorch-style code shows the core mathematical operations and shape flow for scaled dot-product attention. It is a teaching skeleton, not a complete reproduction of the 2017 model: it omits projections, head splitting, dropout, residual branches, normalization, padding-mask construction and output projection.

```
import math
import torch

def scaled_dot_product_attention(q, k, v, mask=None):
    """
    q: [batch, heads, query_len, head_dim]
    k: [batch, heads, key_len, head_dim]
    v: [batch, heads, key_len, value_dim]
    mask: broadcastable to [batch, heads, query_len, key_len];
        True means the position is blocked in this example.
    """
    scores = torch.matmul(q, k.transpose(-2, -1))
    scores = scores / math.sqrt(q.size(-1))

    if mask is not None:
        scores = scores.masked_fill(mask, float("-inf"))

    weights = torch.softmax(scores, dim=-1)
    output = torch.matmul(weights, v)
    return output, weights
```

Implementation note: mask conventions differ across APIs. Some APIs use True to mean "allowed," others use True to mean "blocked"; some expect additive masks rather than booleans. Do not copy a mask into another API without checking its semantics. For half-precision computation, use well-tested framework kernels for numerical behavior and edge cases.

26.1 Useful assertions for a test

```
assert q.shape[-1] == k.shape[-1]      # query/key feature widths match
assert k.shape[-2] == v.shape[-2]      # key/value sequence lengths match
assert weights.shape[-1] == v.shape[-2]
```

```
assert output.shape[-2] == q.shape[-2]  # one output per query position
```

A practical test should also check that valid attention-weight rows sum to approximately one, that masked entries receive zero or near-zero probability, and that output values are finite. An all-masked row should be handled deliberately rather than assumed to work in every implementation.

27. Engineering debugging checklist

Check	Question	Failure symptoms
Tokenizer	Are token IDs correct and from the intended vocabulary?	Unexpected sequence length, unknown IDs, mismatched embeddings.
Embedding lookup	Do IDs index the expected rows?	Wrong shapes or nonsensical input representations.
Q/K/V	Are projection and head shapes consistent?	Matrix multiplication errors or wrong axis ordering.
QK^T	Is the key tensor transposed along the last two dimensions?	Score tensor has unexpected shape.
Scaling	Is the score divided by sqrt(head_dim)?	Attention distribution or gradients differ from the intended operation.
Softmax axis	Does each query normalize over keys?	Rows/columns sum to one along the wrong axis.
Mask	Are future target positions and padding handled correctly?	Information leakage or attention to padding.
Residual	Does sublayer output match the residual branch shape?	Addition errors or accidental broadcasts.
LayerNorm	Is normalization applied over the intended feature dimension?	Unexpected statistics or training instability.
Shifted targets	Does each target position predict the correct next token?	Off-by-one target errors or leakage.
Generation	Is EOS handled, and is the maximum length respected?	Infinite loops, premature stops or overlong outputs.

28. Strengths, trade-offs and limitations

28.1 Strengths introduced or emphasized

- Parallelizable position-wise computation during training, compared with recurrent sequence chains.
- Direct interactions between distant positions within a self-attention layer.
- Multiple learned attention projections that allow several representation subspaces to contribute.
- A modular architecture expressible with matrix operations and reusable blocks.
- Strong machine-translation results and, in the longer arXiv version, encouraging parsing results.

28.2 Trade-offs

- Full self-attention scales quadratically with sequence length in its score matrix.
- Autoregressive generation remains sequential across output tokens.
- Position information must be injected or otherwise represented because attention alone does not encode order.
- Large models require substantial data, compute, memory and optimization work.
- The 2017 encoder-decoder model is not the same as the decoder-only model family now widely used for generative language models.

28.3 Engineering decision questions

- How long are the inputs and outputs, and is full attention affordable at those lengths?
- What are B, N, D, H and D_h, and what are the resulting tensor sizes?
- Does the model require cross-attention, causal masking, padding masks or all of these?
- Is the goal to reproduce the historical paper or to build a modern production system?
- Are tokenizer behavior, target shifting, optimizer schedule, normalization order and decoding settings aligned with the intended reproduction?

PART VI

Historical context, modern Transformer families and mastery

A guided progression from intuition to implementation

29. Original Transformer, BERT, GPT and T5

The original Transformer introduced an encoder-decoder sequence-transduction architecture. Later models reused Transformer blocks but selected different parts of the architecture and different training objectives. The structural comparison below is deliberately high-level; actual objectives, data, scale and fine-tuning procedures differ substantially. [1, 3, 4, 5]

Model family	Typical structure	Common training/use pattern	Historical distinction
Original Transformer (2017)	Encoder + decoder	Sequence-to-sequence translation	The architecture studied in this report.
BERT (2018)	Encoder-oriented	Bidirectional pretraining followed by task fine-tuning	Learns representations using left and right context under its pretraining objective.
GPT (2018 and later family)	Decoder-style causal Transformer	Autoregressive language modeling and generation	Predicts tokens conditioned on a preceding prefix.
T5 (2020)	Encoder + decoder	Text-to-text transfer for multiple NLP tasks	Frames input and output tasks as text-to-text transformations.

29.1 BERT

BERT stands for Bidirectional Encoder Representations from Transformers. Devlin and colleagues introduced an encoder-oriented approach that pretrains bidirectional representations and then fine-tunes for downstream tasks. It is a later development, not a component of the original 2017 system. [3]

29.2 GPT

The 2018 GPT work, “Improving Language Understanding by Generative Pre-Training,” uses a Transformer decoder architecture for autoregressive language modeling followed by task adaptation. Later GPT systems grew into a broad family of generative models, but those later scale and alignment methods should not be retroactively credited to the 2017 paper. [4]

29.3 T5

T5, “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” uses an encoder-decoder Transformer and frames many language tasks as text-to-text problems. For example, translation maps one text to another, summarization maps a document to a summary, and question answering maps a question-plus-context input to an answer. It is later work. [5]

29.4 What modern LLMs retain—and what changed

Many Transformer language models retain the broad pattern of learned token representations, attention, nonlinear transformations, residual paths and repeated layers. Later work changed positional representations, normalization arrangements, activation functions, attention kernels and caching, pretraining corpora and objectives, distributed training, instruction tuning, preference optimization, retrieval and tool use. These are evolutions of the broader Transformer ecosystem, not all elements of “Attention Is All You Need.”

30. Common conceptual mistakes

Misunderstanding	Correction
“A token is a word.”	A token can be a word, subword, punctuation mark, byte-level unit or special token.
“Token ID 81 means the model knows 81 things about cats.”	The ID is an index. The embedding is the learned vector.

Misunderstanding	Correction
“Attention chooses exactly one important word.”	Attention usually forms a weighted combination of multiple value vectors.
“Q is literally a question.”	Q is a learned projection used in compatibility calculation.
“The Transformer is just the attention formula.”	The formula is one core operation inside an encoder-decoder architecture with many other parts.
“The original Transformer is decoder-only.”	The original 2017 architecture has both encoder and decoder stacks.
“Attention was invented in 2017.”	Earlier models already used attention; this paper made it the central mechanism of the sequence-transduction architecture.
“Every attention head is one grammar rule.”	Some heads display linguistic patterns, but head functions are learned and not guaranteed to map cleanly to rules.
“Attention weights fully explain reasoning.”	Weights describe one information-mixing operation, not necessarily the complete reasoning process.
“The Transformer generates the entire target sequence at once.”	Training target positions can be computed in parallel under a mask; standard autoregressive generation is sequential.
“Sinusoidal positions were proven superior.”	The paper reports nearly identical results to learned positions in its ablation.
“Modern LLMs are exactly the 2017 architecture.”	They share foundations but commonly differ in architecture details, training, scale and inference engineering.

31. Recommended learning path

For an absolute beginner, build the concepts in this order instead of memorizing formulas prematurely.

Step	Topic	Practice task
1	Vectors and dimensions	Explain why [0.2, 0.1, 0.5] has three dimensions.
2	Matrix multiplication	Calculate a 2 × 3 matrix multiplied by a 3 × 2 matrix.
3	Dot products	Compute [1,2] · [3,4] and explain what the result is.
4	Softmax	Convert scores [2,1,0] to approximate probabilities.
5	Embeddings	Describe the difference between a token ID and its embedding vector.
6	Training	Explain input → prediction → loss → gradient → update.
7	RNNs and CNNs	Draw where sequential dependency or local receptive fields arise.
8	Scaled dot-product attention	Reproduce the three-token toy example in Section 9.
9	Multi-head attention	Track shape changes from [B,N,D] to [B,H,N,D_h] and back.
10	Encoder and decoder	Draw self-attention, causal self-attention and cross-attention separately.
11	PyTorch	Implement attention and verify shapes, masking and finite outputs.

Step	Topic	Practice task
12	Paper reproduction	Match the historical model settings and distinguish training from decoding.
13	Later models	Compare encoder-oriented BERT, decoder-style GPT and encoder-decoder T5.
14	Modern LLM systems	Study pretraining, inference, retrieval, tool use and evaluation as later topics.

32. Self-test: 30 questions with answer guide

1. What is a token?

Answer: A tokenizer-defined piece of text processed as one unit; it is not necessarily a whole word.

2. What is a token ID?

Answer: An integer index identifying a token in the vocabulary.

3. What is an embedding?

Answer: A learned vector selected by token ID.

4. What does d_model represent?

Answer: The width of the main token representations; 512 in the original base model.

5. What is the difference between training and inference?

Answer: Training updates parameters using targets and gradients; inference uses fixed trained parameters to produce outputs.

6. What does a loss measure?

Answer: A numerical penalty for the model's predictions under the chosen objective.

7. What does backpropagation calculate?

Answer: Gradients of the loss through the computational graph.

8. What does Adam do?

Answer: Updates parameters using gradients and moving estimates based on them.

9. What is the attention equation?

Answer: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\,V$.

10. What are Q, K and V?

Answer: Learned query, key and value projections of representations.

11. Why compute QK^T?

Answer: To get compatibility scores for every query-key pair.

12. Why transpose K?

Answer: It aligns the feature axis for multiplication and produces an $n \times n$ query-key score matrix.

13. Why divide by $\sqrt{d_k}$?

Answer: To control score scale as the key dimension grows and avoid overly saturated softmax under the paper's assumptions.

14. Why use softmax?

Answer: To convert scores into nonnegative normalized weights that sum to one over keys.

15. Why multiply by V?

Answer: To take the weighted sum of value vectors using the attention weights.

16. What is self-attention?

Answer: Q, K and V are derived from the same sequence representation.

17. What is cross-attention?

Answer: Queries come from one stream, while keys and values come from another; in the original decoder they come from encoder output.

18. Why multiple heads?

Answer: To provide several learned projection spaces and parallel attention patterns.

19. What is positional encoding for?

Answer: It provides the model with explicit information about token positions/order.

20. What does the original encoder layer contain?

Answer: Multi-head self-attention and a position-wise FFN, each with residual addition and LayerNorm.

21. What does the decoder add?

Answer: Masked self-attention, encoder-decoder attention, and an FFN, each wrapped with residual and normalization.

22. Why causal masking?

Answer: To stop a target position from using future target tokens it would not know during generation.

23. What is an FFN?

Answer: Two linear transformations with ReLU between them in the original paper; it transforms each position independently.

24. What does a logit mean?

Answer: A raw vocabulary score before softmax.

25. What is BLEU?

Answer: A reference-overlap metric for machine translation, not a complete measure of understanding.

26. What are the base model's key dimensions?

Answer: 6 encoder layers, 6 decoder layers, d_model=512, d_ff=2048, 8 heads, d_k=d_v=64.

27. Why can training be parallelized?

Answer: The target sequence is known and a causal mask prevents future-token leakage while positions are computed together.

28. Why is generation sequential?

Answer: Each next token conditions on the prefix generated so far.

29. What is full attention's sequence-length scaling?

Answer: The score matrix is N × N, so core full self-attention has quadratic sequence-length scaling.

30. How do BERT, GPT and T5 differ structurally?

Answer: BERT is encoder-oriented, GPT is decoder-style causal, and T5 is encoder-decoder text-to-text; all are later developments.

APPENDICES

Reference tables, formula sheet, glossary and source list

A guided progression from intuition to implementation

Appendix A. Original base and big model settings

Setting	Transformer base	Transformer big	Notes
Encoder layers N	6	6	Same number of layers.
Decoder layers N	6	6	Same number of layers.
d_model	512	1024	Representation width.
d_ff	2048	4096	FFN hidden width.
Attention heads h	8	16	Parallel heads.
d_k, d_v	64, 64	64, 64	Per-head dimensions in the reported configurations.
Dropout	0.1	0.3 in Table 3 big setting; 0.1 for EN→FR big run	The English-French big-model training uses 0.1.
Label smoothing	0.1	0.1	Reported training setting.
Training steps	100K	300K	Paper's historical schedule.
Parameters	About 65M	About 213M	As reported by the paper.
Step time	About 0.4 s	About 1.0 s	On one machine with 8 NVIDIA P100 GPUs.
Training time	About 12 hours	About 3.5 days	Historical reported times.

Appendix B. Formula sheet

Concept	Formula	Plain-English meaning
Query/key/value projections	Q = XW_Q; K = XW_K; V = XW_V	Create three learned views of the representations.
Scaled dot-product attention	softmax(QK^T / √d_k) V	Compare; scale; normalize; combine.
Multi-head attention	Concat($head_1$, …, head_h) W_O	Run heads in parallel and project their concatenation.
Original FFN	max(0, $xW_1 + b_1$) $W_2 + b_2$	Expand features, apply ReLU, compress back.
Post-norm sublayer	LayerNorm(x + Sublayer(x))	Residual add followed by normalization.
Sinusoidal position (even)	sin(pos / 10000^(2i/d_model))	Position-specific sine values at multiple wavelengths.
Sinusoidal position (odd)	cos(pos / 10000^(2i/d_model))	Position-specific cosine values at multiple wavelengths.
Learning-rate schedule	d_model^(−0.5) × min(step^(−0.5), step × warmup^(−1.5))	Linear warmup, then inverse-square-root decay.
Simplified token loss	−log(p_correct)	Penalize low probability assigned to the correct token.
F1	2PR / (P+R)	Harmonic mean of precision P and recall R.

Appendix C. Glossary

Term	Definition
Attention	A function that uses query-key compatibility to calculate weights and then forms a weighted sum of value vectors.
Autoregressive	Generating a sequence one element at a time, conditioned on earlier elements.
Batch	A group of training examples processed together.
BPE	Byte Pair Encoding, a subword tokenization approach.
Causal mask	A mask that blocks a target position from using future target positions.
Checkpoint	A saved snapshot of model parameters during or after training.
Cross-attention	Attention whose queries and keys/values come from different representation streams.
Decoder	The stack that generates target representations and output tokens.
Embedding	A learned vector representation indexed by a token ID.
Encoder	The stack that builds contextual representations of the source sequence.
FFN	Feed-forward network applied independently at each position in a Transformer layer.
Gradient	A derivative or collection of derivatives indicating how the loss changes with parameters.
Layer normalization	Normalization across features within a representation, followed by learned scale and shift.
Logit	A raw score before conversion to probabilities, usually by softmax.
Mask	A constraint that disallows attention or loss computation at selected positions.
Multi-head attention	Several attention calculations with separate learned projections, followed by concatenation and projection.
Perplexity	An exponential transformation of average negative log-likelihood, used to summarize predictive probability.
Positional encoding	A vector signal added to token embeddings to represent position.
Residual connection	A direct addition path from a sublayer input to its output.
Self-attention	Attention in which Q, K and V come from the same sequence representation.
Softmax	A function that converts scores into positive weights summing to one along an axis.
Token	A tokenizer-defined text unit.
Vocabulary	The set of token IDs a tokenizer/model supports.
Weight sharing	Using the same parameter matrix in more than one model component.
BLEU	A machine-translation metric based on n-gram overlap and brevity penalty.

Appendix D. References and primary sources

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). “Attention Is All You Need.” Advances in Neural Information Processing Systems 30 (NeurIPS 2017). This report uses the published proceedings PDF for the main published benchmark values.

Proceedings PDF: https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

NeurIPS paper page: https://papers.neurips.cc/paper/7181-attention-isall-you-need

[2] Vaswani, A. et al. “Attention Is All You Need.” arXiv:1706.03762, current version v7 (2023). The longer version includes the English constituency parsing experiment and attention visualizations. Note the 41.8 abstract versus 41.0 results-text discrepancy discussed in this report.

arXiv record: https://arxiv.org/abs/1706.03762

arXiv PDF: https://arxiv.org/pdf/1706.03762

[3] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” arXiv:1810.04805.

https://arxiv.org/abs/1810.04805

[4] Radford, A. et al. (2018). “Improving Language Understanding by Generative Pre-Training.” OpenAI research paper.

https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

[5] Raffel, C. et al. (2020). “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.” Journal of Machine Learning Research 21(140), 1–67.

https://www.jmlr.org/papers/v21/20-074.html

[6] Bahdanau, D., Cho, K., & Bengio, Y. (2014). “Neural Machine Translation by Jointly Learning to Align and Translate.” arXiv:1409.0473. Cited for background on attention in sequence-to-sequence models before the Transformer.

https://arxiv.org/abs/1409.0473

[7] Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). “BLEU: a Method for Automatic Evaluation of Machine Translation.” Proceedings of ACL 2002.

https://aclanthology.org/P02-1040/

[8] Sennrich, R., Haddow, B., & Birch, A. (2016). “Neural Machine Translation of Rare Words with Subword Units.” Proceedings of ACL 2016.

https://aclanthology.org/P16-1162/

Appendix E. Final synthesis

The whole paper in one paragraph

The original Transformer converts token IDs to learned embeddings, adds positional information, and repeatedly applies multi-head attention plus position-wise feed-forward networks inside residual-and-normalized encoder and decoder stacks. Scaled dot-product attention computes $\text{softmax}(QK^T / \sqrt{d_k})V$. Encoder self-attention contextualizes the source, masked decoder self-attention preserves autoregressive causality, and cross-attention lets the decoder use source representations. The design removed recurrent and convolutional sequence layers from the core architecture, made training more parallelizable, and achieved strong 2017 translation results. The attention equation is the center of the story, but the complete architecture and training recipe are what made the paper work.

A final mental model:

```
Token ID        → selects a learned embedding
Position        → provides sequence-order information
Attention       → mixes information across permitted positions
Feed-forward    → transforms each position independently
Residual + Norm → support a stable deep stack
Encoder         → represents the source sequence
Decoder         → predicts target tokens without seeing the future
Linear + Softmax→ turns hidden states into token probabilities
```

If a learner can explain the token-to-embedding path, derive the attention equation, reproduce the toy calculation, trace the encoder and decoder, explain causal masking, and describe why training can be parallel while generation remains sequential, they have understood the essential mechanics of “Attention Is All You Need.”